

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO MÔN HỌC
CAPTCHA Reader VLM
Đấu Trường YOLO vs Donut vs TrOCR

Đề tài: Xây dựng pipeline đọc CAPTCHA và so sánh ba mô hình AI
Môn học: Nhập môn học máy và khai phá dữ liệu
Giảng viên: Ngô Văn Linh

STT	Họ và tên thành viên	MSSV
1	Hoàng Minh Khôi	20230042
2	Nguyễn Thanh Tú	20230075
3	Đỗ Trường Giang	20235066

Hà Nội, tháng 6 năm 2026

Lời cảm ơn

Nhóm xin chân thành cảm ơn quý thầy cô đã hướng dẫn, cung cấp kiến thức và tạo điều kiện hoàn thành đồ án. Nhóm cũng ghi nhận các tài liệu mã nguồn mở từ Ultralytics (YOLO), Hugging Face (Donut, TrOCR), Albumentations và cộng đồng nghiên cứu Vision-Language Model.

Tóm tắt

Từ khóa: CAPTCHA, YOLO, Donut, TrOCR, OCR, Vision-Language Model, sinh dữ liệu tổng hợp.

Báo cáo trình bày hệ thống **Captcha_Reader** — một pipeline học máy end-to-end nhằm giải CAPTCHA chữ bị méo bằng cách so sánh ba hướng tiếp cận: **YOLO11x** (phát hiện từng ký tự), **Donut** (mô hình Vision-Language với token tác vụ tùy chỉnh), và **TrOCR** (OCR seq2seq của Microsoft).

Hệ thống giải quyết bài toán thiếu dữ liệu bằng module **CaptchaGenerator** sinh 10 000 ảnh CAPTCHA tổng hợp kèm nhãn YOLO, áp dụng méo ảnh đa tầng (sóng, xoay, chữ rộng, chồng lấn) và **CaptchaAugmenter** dựa trên Albumentations. Ba mô hình được huấn luyện trên cùng dataset, đánh giá bằng CER và Exact Match, theo dõi qua Weights & Biases. Ứng dụng Gradio (`app.py`) cho phép upload ảnh và so sánh kết quả ba mô hình trực quan.

Mục lục

Lời cảm ơn	1
Tóm tắt	2
1 Giới thiệu	7
1.1 Bối cảnh và vấn đề	7
1.2 Mục tiêu đề án	7
1.3 Phạm vi và giới hạn	7
1.4 Cấu trúc báo cáo	8
2 Cơ sở lý thuyết	9
2.1 CAPTCHA và OCR	9
2.2 YOLO — Object Detection	9
2.3 Donut — Document Vision-Language Model	9
2.4 TrOCR — Transformer OCR	9
2.5 Các chỉ số đánh giá	9
3 Kiến trúc hệ thống	11
3.1 Tổng quan kiến trúc	11
3.2 Công nghệ sử dụng	11
3.3 Danh mục file mã nguồn	12
3.4 Cấu hình tập trung (<code>config.py</code>)	12
4 Sinh dữ liệu CAPTCHA tổng hợp	13
4.1 Lớp <code>CaptchaGenerator</code>	13
4.1.1 Khởi tạo	13
4.1.2 Hàm <code>generate_data()</code>	13
4.1.3 Hàm <code>_distort_single_char_cv2()</code>	14
4.2 Lớp <code>DatasetBuilder</code>	14
4.2.1 Chiến lược Train vs Validation	14
4.2.2 Hàm <code>build_yolo_labels()</code>	14
4.2.3 Cấu trúc thư mục dataset	14
5 Augmentation và tiền xử lý	15
5.1 Lớp <code>CaptchaAugmenter</code>	15
5.1.1 Custom transforms	15
5.1.2 Pipeline tổng hợp	15
5.1.3 Augmentation on-the-fly khi train	15

6	Mô hình YOLO11x	17
6.1	Ý tưởng	17
6.2	Cấu hình dataset (<code>dataset.yaml</code>)	17
6.3	Custom Dataset và Trainer	17
6.3.1	<code>CustomCaptchaDataset</code>	17
6.3.2	<code>CustomTrainer</code>	17
6.3.3	Tắt augmentation mặc định của YOLO	18
6.4	Hyperparameters huấn luyện	18
6.5	Callback đánh giá CER tùy chỉnh	18
6.6	Workaround <code>torchaudio</code>	18
6.7	Inference (<code>app.py</code> — <code>run_yolo</code>)	18
7	Mô hình Donut	19
7.1	Kiến trúc và pre-trained	19
7.2	Token tác vụ tùy chỉnh	19
7.3	Dataset <code>CaptchaDonutDataset</code>	19
7.4	Metrics	19
7.5	Hyperparameters	20
7.6	Patch Swin Encoder	20
7.7	Inference	20
8	Mô hình TrOCR	21
8.1	Kiến trúc	21
8.2	Dataset <code>CaptchaTrOCRDataset</code>	21
8.3	<code>Seq2SeqTrainer</code>	21
8.4	Generation config	21
8.5	<code>RestrictVocabLogitsProcessor</code> (<code>test_trocr.py</code>)	22
8.6	Hyperparameters	22
9	Ứng dụng Inference và Demo	23
9.1	Ứng dụng Gradio (<code>app.py</code>)	23
9.1.1	Tiền xử lý ảnh đầu vào	23
9.1.2	Giao diện	23
9.2	Hàm <code>clean_text()</code> (<code>utils.py</code>)	23
9.3	Callback trực quan WandB	24
10	Đánh giá thực nghiệm	25
10.1	Thiết lập thí nghiệm	25
10.2	Bảng so sánh hyperparameters	25
10.3	Kết quả (điền sau khi train)	25
10.4	Phân tích định tính	26
11	Khó khăn, giải pháp và hướng phát triển	27
11.1	Khó khăn gặp phải	27
11.2	Hướng phát triển	27
	Kết luận	28
	Tài liệu tham khảo	29

A Phụ lục: Hướng dẫn cài đặt và chạy	30
A.1 Cài đặt môi trường	30
A.2 Lệnh chạy chính	30

Danh sách bảng

3.1	Stack công nghệ	11
3.2	Phân công module theo trách nhiệm (SRP)	12
3.3	Tham số chính trong <code>config.py</code>	12
5.1	Các transform tùy chỉnh	15
6.1	Cấu hình train YOLO (<code>YOLO.py</code>)	18
7.1	Cấu hình train Donut	20
8.1	Cấu hình train TrOCR	22
10.1	So sánh cấu hình huấn luyện ba mô hình	25
10.2	Kết quả đánh giá trên tập validation	25
11.1	Khó khăn và giải pháp trong quá trình triển khai	27

Chương 1

Giới thiệu

1.1 Bối cảnh và vấn đề

CAPTCHA (*Completely Automated Public Turing test to tell Computers and Humans Apart*) là lớp bảo vệ phổ biến trên web, yêu cầu người dùng nhận dạng chuỗi ký tự bị méo. Với sự phát triển của học sâu, việc đọc CAPTCHA trở thành bài toán thực nghiệm quan trọng để đánh giá khả năng nhận dạng ký tự trong điều kiện nhiễu cao.

Các thách thức chính:

- **Thiếu dữ liệu:** CAPTCHA thật khó thu thập hợp pháp và đủ đa dạng.
- **Biến dạng phức tạp:** Chữ bị sóng, xoay, chồng lấn, gạch ngang, nhiễu nền.
- **Đa hướng tiếp cận:** Detection từng ký tự (YOLO) vs. OCR end-to-end (TrOCR) vs. VLM (Donut) — chưa có so sánh thống nhất trên cùng pipeline.

1.2 Mục tiêu đề án

1. Xây dựng công cụ sinh dataset CAPTCHA tổng hợp với nhãn YOLO và OCR.
2. Huấn luyện và fine-tune ba mô hình: YOLO11x, Donut, TrOCR.
3. Thiết kế hệ thống đánh giá thống nhất (CER, Exact Match, mAP).
4. Triển khai giao diện demo Gradio so sánh inference thời gian thực.

1.3 Phạm vi và giới hạn

- Bộ ký tự tùy chỉnh 56 ký tự (VOCAB), loại bỏ ký tự dễ nhầm (0/0, 1/I/1).
- CAPTCHA dạng chữ ngang, 5–8 ký tự, kích thước 220×60 pixel.
- Inference mặc định trên CPU (`app.py`); huấn luyện tối ưu cho GPU (A100).
- Không triển khai REST API production; tập trung nghiên cứu và demo.

1.4 Cấu trúc báo cáo

Chương 2 trình bày cơ sở lý thuyết. Chương 3 mô tả kiến trúc hệ thống. Chương 4–5 chi tiết sinh dữ liệu và augmentation. Chương 6–8 phân tích ba mô hình và mã nguồn huấn luyện. Chương 9–10 trình bày inference, đánh giá và kết luận.

Chương 2

Cơ sở lý thuyết

2.1 CAPTCHA và OCR

OCR (*Optical Character Recognition*) chuyển ảnh chữ thành text. Với CAPTCHA, OCR truyền thống (Tesseract) thường thất bại do nhiễu. Học sâu cho phép học biểu diễn trực tiếp từ dữ liệu.

2.2 YOLO — Object Detection

YOLO (*You Only Look Once*) phát hiện đối tượng trong một lần forward pass. Với CAPTCHA, mỗi ký tự là một class; chuỗi kết quả được ghép bằng cách sắp xếp bounding box theo tọa độ x tăng dần.

Ưu điểm: Modular, có thể debug từng ký tự, metric mAP chuẩn.

Nhược điểm: Phụ thuộc chất lượng detection và thuật toán sắp xếp; khó với ký tự chồng lấn nặng.

2.3 Donut — Document Vision-Language Model

Donut (Kim et al., 2022) kết hợp Swin Transformer encoder và BART decoder, đọc tài liệu end-to-end không cần OCR pipeline riêng. Dự án mở rộng bằng token tác vụ `<s_captcha>` và `</s_captcha>` để fine-tune cho CAPTCHA.

Input: Ảnh $\rightarrow$ pixel values.

Output: Chuỗi token `<s_captcha>TEXT</s_captcha>`.

2.4 TrOCR — Transformer OCR

TrOCR (Li et al., 2021) dùng Vision Encoder (ViT) + Text Decoder (RoBERTa), pre-trained trên dữ liệu in ấn. Phiên bản `trocr-base-printed` phù hợp CAPTCHA dạng chữ in.

2.5 Các chỉ số đánh giá

- **CER (Character Error Rate):** Tỷ lệ ký tự sai so với chuỗi gốc, dùng thư viện `evaluate`.

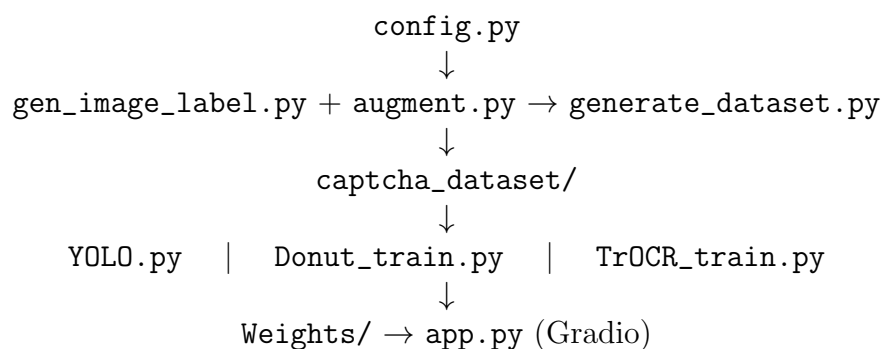
- **Exact Match (EM):** Tỷ lệ chuỗi dự đoán trùng hoàn toàn chuỗi ground truth.
- **mAP50 / mAP50-95:** Metric detection của YOLO (IoU 0.5 và trung bình 0.5–0.95).

Chương 3

Kiến trúc hệ thống

3.1 Tổng quan kiến trúc

Dự án có cấu trúc phẳng (flat repository), không chia package con. Luồng xử lý chính:



3.2 Công nghệ sử dụng

Bảng 3.1: Stack công nghệ

Lớp	Công nghệ	Phiên bản (lock)
Ngôn ngữ	Python	≥ 3.12
Quản lý gói	uv + pyproject.toml	—
Deep Learning	PyTorch, Transformers	5.12.1
Detection	Ultralytics YOLO	8.4.72
Augmentation	Albumentations	2.0.8
Giao diện	Gradio	6.19.0
Theo dõi thí nghiệm	Weights & Biases	0.27.2
Xử lý ảnh	OpenCV, Pillow	—

3.3 Danh mục file mã nguồn

Bảng 3.2: Phân công module theo trách nhiệm (SRP)

File	Trách nhiệm
config.py	Cấu hình tập trung: đường dẫn, charset, tham số méo ảnh
gen_image_label.py	CaptchaGenerator: sinh 1 cặp (ảnh, nhãn YOLO)
augment.py	CaptchaAugmenter: pipeline Albumentations + custom transforms
generate_dataset.py	DatasetBuilder: I/O hàng loạt, convert nhãn
marker.py	BBoxMarker: vẽ bbox debug
utils.py	clean_text(): lọc token Donut
YOLO.py	Custom dataset/trainer YOLO + callback CER
Donut_train.py	Dataset + Trainer Donut
TrOCR_train.py	Dataset + Seq2SeqTrainer TrOCR
app.py	Inference Gradio 3 mô hình
modeling_donut_swin.py	Patch Swin encoder cho Donut (964 dòng)

3.4 Cấu hình tập trung (config.py)

Toàn bộ siêu tham số được khai báo tại config.py:

Bảng 3.3: Tham số chính trong config.py

Tham số	Giá trị	Ý nghĩa
NUM_IMAGES	10 000	Tổng số ảnh sinh
TRAIN_RATIO	0.9	90% train, 10% val
IMG_W $\times$ IMG_H	220 $\times$ 60	Kích thước ảnh CAPTCHA
MIN_CHARS – MAX_CHARS	5 – 8	Số ký tự mỗi ảnh
STEP_MIN – STEP_MAX	–13 – 8	Khoảng cách ký tự (âm = chồng lấn)
HOLLOW_PROB	0.3	Xác suất chữ rỗng (hollow)
FONT_SIZE	40	Cỡ font khi render
CHAR_CANVAS_SIZE	(100, 100)	Canvas RGBA trung gian mỗi ký tự

Bộ ký tự:

```

1 VOCAB      = "23456789ABCDEFGHIJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxy
   "         # 56 ký tự
2 CHARSET    = "23456789ABCDEFGHIJKLMNPQRSTUVWXYZ"
               # 32 ký tự (YOLO)

```

Chương 4

Sinh dữ liệu CAPTCHA tổng hợp

4.1 Lớp CaptchaGenerator

File `gen_image_label.py` định nghĩa class `CaptchaGenerator` với nguyên tắc **SRP**: chỉ sinh một cặp (ảnh, nhãn) trên RAM, không ghi đĩa.

4.1.1 Khởi tạo

Constructor tải sẵn 10 font TTF từ `FONT_PATHS` (Arial, Calibri, Consola, Courier, Georgia, Segoe UI, Tahoma, Times, Trebuchet, Verdana) vào bộ nhớ để tối ưu tốc độ khi sinh hàng loạt.

4.1.2 Hàm `generate_data()`

Quy trình sinh một ảnh CAPTCHA:

1. Tạo nền RGB ngẫu nhiên (màu sáng: 200–255).
2. Chọn ngẫu nhiên $n \in [5, 8]$ ký tự từ VOCAB.
3. Với mỗi ký tự:
 - Chọn font ngẫu nhiên, vẽ lên canvas RGBA 100×100 .
 - Màu chữ tối (0–100), stroke width 1–2 để chữ béo hơn.
 - Scale ngẫu nhiên $[0.7, 1.2]$, méo qua `_distort_single_char_cv2()`.
 - Shear (xiên) $[-0.3, 0.3]$, xoay $[-35, 35]$.
 - Paste lên ảnh nền tại vị trí (`current_x`, `current_y`) ngẫu nhiên theo chiều dọc.
 - Tính bbox chuẩn hóa YOLO (`class`, `xc`, `yc`, `w`, `h`) với margin 0.002 chống sai số float.
 - Cập nhật `current_x += cw + step`, với `step` $\in [-13, 8]$ (cho phép chồng ký tự).
4. Trả về (`base_image`, `labels_content`).

4.1.3 Hàm `_distort_single_char_cv2()`

Đây là lõi méo ảnh từng ký tự, gồm 4 giai đoạn:

(1) **Hollow text (chữ rỗng)**: Với xác suất 30%, tạo viền chữ bằng phép dilate/erode: $\text{hollow} = \text{dilate}(\text{base}) - \text{erode}(\text{base})$. Kernel thích ứng theo kích thước chữ (tối đa 3px).

(2) **Morphology**: Với xác suất 70%, dilate hoặc erode với kernel dọc/ngang ngẫu nhiên.

(3) **Wave distortion**: Ánh xạ lưới tọa độ bằng hàm sin/cos:

$$x' = x + s_x \sin(y/\lambda + \phi_x) \quad (4.1)$$

$$y' = y + s_y \cos(x/\lambda + \phi_y) \quad (4.2)$$

với $s \in [3, 6]$, $\lambda \in [9, 20]$. Dùng `cv2.remap` với interpolation cubic.

4.2 Lớp DatasetBuilder

File `generate_dataset.py` quản lý I/O hàng loạt.

4.2.1 Chiến lược Train vs Validation

- **Train (9 000 ảnh)**: Lưu ảnh *phôi gốc* từ generator. Augmentation được áp dụng **on-the-fly** lúc train.
- **Val (1 000 ảnh)**: Áp dụng `CaptchaAugmenter` **trước khi lưu** để phân phối val bám sát nhiều thực tế, cố định trên đĩa.

4.2.2 Hàm `build_yolo_labels()`

Chuyển nhãn từ `OCR_labels/` (56 class, VOCAB) sang `labels/` (32 class, CHARSET):

1. Đọc `class_id` từ file OCR.
2. Tra `VOCAB[old_id] → ký tự thật → ép hoa`.
3. Ánh xạ sang `CHARSET.index(upper_char) → new_id`.
4. Giữ nguyên 4 tọa độ bbox.

Lưu ý: Trong `__main__`, `builder.build()` bị comment; mặc định chỉ chạy `build_yolo_labels()`

4.2.3 Cấu trúc thư mục dataset

```
captcha_dataset/
  train/images/          # PNG captcha_000000.png ...
  train/OCR_labels/      # Nhan 56 class (Donut, TrOCR)
  train/labels/          # Nhan 32 class (YOLO)
  val/images/
  val/OCR_labels/
  val/labels/
```

Chương 5

Augmentation và tiền xử lý

5.1 Lớp CaptchaAugmenter

Định nghĩa trong `augment.py`, dùng Albumentations với `BboxParams(format='yolo')` để đồng bộ bbox khi biến đổi ảnh.

5.1.1 Custom transforms

Bảng 5.1: Các transform tùy chỉnh

Class	Mô tả
<code>RandomNoiseLines</code>	2–6 đường thẳng ngẫu nhiên, độ dày 1–3px
<code>RandomNoiseDots</code>	35–40 chấm tròn, bán kính 2–4px
<code>RandomRegionBlur</code>	1–6 vùng blur cục bộ, kernel 3–5
<code>StrikethroughLine</code>	1–5 đường gạch ngang xuyên ảnh
<code>RandomQuadInvert</code>	Đảo màu trong vùng tứ giác ngẫu nhiên

5.1.2 Pipeline tổng hợp

```
1 A.Compose([
2     A.GridDistortion(num_steps=5, distort_limit=0.2, p=0.3),
3     A.OneOf([RandomNoiseLines, StrikethroughLine, RandomRegionBlur], p=0.7),
4     A.OneOf([RandomQuadInvert, RandomNoiseDots, A.GaussNoise], p=0.6),
5     A.ColorJitter(brightness=0.3, contrast=0.3, p=0.5),
6     A.Blur(blur_limit=2, p=0.2),
7 ], bbox_params=A.BboxParams(format='yolo', label_fields=['class_labels']))
```

5.1.3 Augmentation on-the-fly khi train

Cả ba script huấn luyện đều gọi `CaptchaAugmenter` trong `__getitem__` (hoặc `get_image_and_label` với YOLO) **chỉ trên tập train**. Tập val không augment thêm (Donut/TrOCR) hoặc

đã augment sẵn khi sinh (val images). Có thể xem trước hiệu ứng bằng lệnh `uv run sample.py`.

Chương 6

Mô hình YOLO11x

6.1 Ý tưởng

YOLO coi mỗi ký tự là một đối tượng thuộc 1 trong 32 class. Sau inference, các box được sắp xếp theo tọa độ x trung tâm tăng dần, ghép `CHARSET[class_id]` thành chuỗi.

6.2 Cấu hình dataset (dataset.yaml)

```
1 path: ./captcha_dataset
2 train: train/images
3 val: val/images
4 names: # 32 class, thu tu khop CHARSET
5   0: '2'
6   1: '3'
7   ...
8   31: 'Z'
```

6.3 Custom Dataset và Trainer

File YOLO.py ghi đè Ultralytics:

6.3.1 CustomCaptchaDataset

Kế thừa `YOLODataset`, override `get_image_and_label()`:

- Gọi hàm gốc lấy ảnh BGR + instances.
- Nếu `augment=True` (train): chuyển BGR→RGB, gọi `CaptchaAugmenter`, cập nhật `instances.bboxes` và `cls`.
- Fallback im lặng nếu augment lỗi (box bị loại).

6.3.2 CustomTrainer

Override `build_dataset()` để YOLO dùng `CustomCaptchaDataset` thay vì dataset mặc định.

6.3.3 Tắt augmentation mặc định của YOLO

Để tránh xung đột với CaptchaAugmenter:

```
1 "mosaic": 0.0, "fliplr": 0.0, "flipud": 0.0,
2 "degrees": 0.0, "mixup": 0.0
```

6.4 Hyperparameters huấn luyện

Bảng 6.1: Cấu hình train YOLO (YOLO.py)

Tham số	Giá trị
Base model	yolo11x.pt
Image size	320
Batch size	128
Epochs	150
Patience	30
WandB project	Captcha_YOLOv11x

6.5 Callback đánh giá CER tùy chỉnh

Hàm `on_fit_epoch_end` chạy sau mỗi epoch:

1. Load checkpoint `trainer.last` bằng `YOLO()`.
2. Duyệt toàn bộ ảnh val, decode ground truth từ `labels/` (sort theo x).
3. Predict, sort box, ghép chuỗi.
4. Tính CER, Exact Match, log thêm mAP50/mAP50-95 lên WandB.

6.6 Workaround torchaudio

YOLO/Ultralytics có thể import `torchaudio` gây lỗi. Mã nguồn mock module rỗng vào `sys.modules['torchaudio']` trước khi import Ultralytics.

6.7 Inference (app.py — run_yolo)

```
1 results = yolo_model(img_pil, verbose=False)[0]
2 for box in results_boxes:
3     cls_id = int(box.cls[0])
4     char_label = id_to_char[cls_id] # CHARSET
5     # Ve box bang OpenCV, lưu x_center de sort
6 pred_boxes.sort(key=lambda item: item[0])
7 final_text = "".join([item[1] for item in pred_boxes])
```

Chương 7

Mô hình Donut

7.1 Kiến trúc và pre-trained

- Base: naver-clova-ix/donut-base
- Encoder: Swin Transformer (patch modeling_donut_swin.py)
- Decoder: BART
- Kích thước ảnh đầu vào: height=128, width=384 (do_align_long_axis=False)

7.2 Token tác vụ tùy chỉnh

Thêm 2 token vào tokenizer và resize embedding decoder:

```
1 processor.tokenizer.add_tokens(["<s_captcha>", "</s_captcha>"])
2 model.decoder.resize_token_embeddings(len(processor.tokenizer))
3 model.config.decoder_start_token_id = id("<s_captcha>")
4 model.config.eos_token_id = id("</s_captcha>")
```

Chuỗi huấn luyện: <s_captcha>TEXT</s_captcha>

7.3 Dataset CaptchaDonutDataset

- Đọc OCR_labels/, gom ký tự thành text_label (ép hoa).
- Augment on-the-fly nếu train.
- Encode: pixel_values + labels (token ids, pad → -100).

7.4 Metrics

Hàm compute_metrics: decode với skip_special_tokens=False, gọi clean_text() để cắt tại </s_captcha> và loại tag.

7.5 Hyperparameters

Bảng 7.1: Cấu hình train Donut

Tham số	Giá trị
Batch size (train/eval)	64
Learning rate	2×10^{-5}
Epochs	100
Precision	bf16
Early stopping patience	5
Best model metric	eval_loss
WandB project	donut-captcha-vlm
Output dir	./tmp_donut_checkpoints

7.6 Patch Swin Encoder

File `modeling_donut_swin.py` (964 dòng) phải copy vào:

`.venv/Lib/site-packages/transformers/models/donut/modeling_donut_swin.py`

Đây là bản Swin Transformer tùy chỉnh cho Donut, không có final layer norm trên hidden states cuối.

7.7 Inference

```

1 task_prompt = "<s_captcha>"
2 decoder_input_ids = tokenizer(task_prompt, ...).input_ids
3 generated_ids = model.generate(pixel_values, decoder_input_ids
4     ...,
5     max_length=12, num_beams=4, early_stopping=True)
text = clean_text(batch_decode(generated_ids, skip_special_tokens=
    False))

```

Chương 8

Mô hình TrOCR

8.1 Kiến trúc

- Base: microsoft/trocr-base-printed
- Vision Encoder: ViT
- Text Decoder: RoBERTa-based
- Không cần prompt đặc biệt; label là chuỗi text thuần

8.2 Dataset CaptchaTrOCRDataset

Tương tự Donut nhưng:

- `labels = tokenize text_label` trực tiếp (không có `<s_captcha>`).
- Dùng `TrOCRProcessor` thay `DonutProcessor`.

8.3 Seq2SeqTrainer

Khác Donut ở các điểm:

- Dùng `Seq2SeqTrainingArguments` + `Seq2SeqTrainer`.
- `predict_with_generate=True` — bắt buộc để eval sinh text thật.
- `metric_for_best_model="cer"` (thực dụng hơn loss).

8.4 Generation config

```
1 model.generation_config.num_beams = 4
2 model.generation_config.max_length = 12
3 model.generation_config.early_stopping = True
4 model.generation_config.no_repeat_ngram_size = 3
5 model.generation_config.length_penalty = 2.0
```

8.5 RestrictVocabLogitsProcessor (test_trocr.py)

Thử nghiệm giới hạn logits chỉ cho phép token thuộc VOCAB, giảm hallucination khi test trên dữ liệu sinh tổng hợp.

8.6 Hyperparameters

Bảng 8.1: Cấu hình train TrOCR

Tham số	Giá trị
Batch size	64
Learning rate	2×10^{-5}
Epochs	100
Dataloader workers	8
Early stopping	patience 5
Best model metric	CER
WandB project	trocr-captcha-vlm

Chương 9

Ứng dụng Inference và Demo

9.1 Ứng dụng Gradio (app.py)

Khi khởi động, nạp đồng thời 3 model từ Weights/:

- best_yolo11x.pt
- best_trocr_model/
- best_donut_model/

Device mặc định: CPU (torch.device("cpu")).

9.1.1 Tiền xử lý ảnh đầu vào

```
1 w, h = input_img.size
2 new_h = int(h * (220 / w))
3 input_img_rescaled = input_img.resize((220, new_h))
```

Chuẩn hóa chiều rộng về 220px (khớp IMG_W trong config).

9.1.2 Giao diện

- Input: upload/paste ảnh PIL.
- Output: ảnh YOLO có bbox + 3 textbox (YOLO, Donut, TrOCR).
- Theme: gr.themes.Soft(), inbrowser=True.

9.2 Hàm clean_text() (utils.py)

Dùng chung cho Donut train/eval/inference:

```
1 def clean_text(text: str) -> str:
2     if "</s_captcha>" in text:
3         text = text.split("</s_captcha>")[0]
4         clean = re.sub(r'<[~>]+>', '', text).strip()
5     return clean
```


9.3 Callback trực quan WandB

Ba file `sample_test*.py` log ảnh mẫu mỗi epoch lên WandB:

- `sample_test.py` — Donut
- `sample_test_TrOCR.py` — TrOCR
- `sample_test_YOLO.py` — YOLO (vẽ box)

Chương 10

Đánh giá thực nghiệm

10.1 Thiết lập thí nghiệm

- Dataset: 10 000 ảnh (9 000 train / 1 000 val), hoặc tải sẵn từ Google Drive.
- Đánh giá trên tập val và thư mục `real_captcha/` (CAPTCHA thực tế).
- Metrics thống nhất: CER, Exact Match; YOLO thêm mAP.

10.2 Bảng so sánh hyperparameters

Bảng 10.1: So sánh cấu hình huấn luyện ba mô hình

Tham số	YOLO	Donut	TrOCR
Loại trainer	CustomTrainer	Trainer	Seq2SeqTrainer
Batch size	128	64	64
Learning rate	(mặc định YOLO)	2×10^{-5}	2×10^{-5}
Epochs	150	100	100
Nhãn train	bbox (32 cls)	<s_captcha>...	plain text
Augment train	on-the-fly	on-the-fly	on-the-fly
Best metric	mAP + CER	eval_loss	CER
Beam search	N/A	4	4

10.3 Kết quả (điền sau khi train)

Bảng 10.2: Kết quả đánh giá trên tập validation

Mô hình	CER	Exact Match	mAP50	Ghi chú
YOLO11x	—	—	—	Detection + sort
Donut	—	—	N/A	VLM + task token
TrOCR	—	—	N/A	OCR end-to-end

Nhóm điền số liệu thực tế từ WandB sau khi hoàn tất huấn luyện.

10.4 Phân tích định tính

- **YOLO:** Mạnh khi ký tự tách rời; yếu khi chồng lẫn nặng (STEP_MIN=-13).
- **Donut:** Linh hoạt với prompt tác vụ; cần patch Swin, cấu hình token phức tạp hơn.
- **TrOCR:** Đơn giản nhất về pipeline; phụ thuộc pre-trained trên chữ in.

Chương 11

Khó khăn, giải pháp và hướng phát triển

11.1 Khó khăn gặp phải

Bảng 11.1: Khó khăn và giải pháp trong quá trình triển khai

Khó khăn	Giải pháp trong mã nguồn
Thiếu dữ liệu	CaptchaGenerator + DatasetBuilder
Bbox lệch sau augment	Albumentations BboxParams + fallback ảnh gốc
Donut lỗi Swin	Patch modeling_donut_swin.py
YOLO import torchaudio	Mock sys.modules['torchaudio']
Donut sinh text thừa	clean_text() cắt tại </s_captcha>
Hai định dạng nhãn	build_yolo_labels() OCR→YOLO

11.2 Hướng phát triển

1. Tăng quy mô dataset lên 100k+ ảnh (comment trong config gợi ý A100).
2. Huấn luyện trên GPU local, hỗ trợ CUDA trong app.py.
3. Ensemble voting từ 3 mô hình.
4. Triển khai FastAPI + Docker cho production.
5. Mở rộng charset, CAPTCHA dạng dọc hoặc đa dòng.

Kết luận

Nhóm đã xây dựng thành công hệ thống **Captcha_Reader** với pipeline hoàn chỉnh từ sinh dữ liệu đến demo inference. Ba mô hình YOLO11x, Donut và TrOCR được triển khai trên cùng nền tảng mã nguồn, chia sẻ module augmentation và metrics, tạo điều kiện so sánh công bằng.

Đóng góp kỹ thuật chính:

- Module sinh CAPTCHA đa tầng méo (wave, hollow, overlap) kèm nhãn YOLO chính xác.
- Pipeline Albumentations bbox-aware với 5 custom transforms.
- Custom YOLO trainer tích hợp augment và callback CER/EM.
- Fine-tune Donut với task token và TrOCR với Seq2SeqTrainer.
- Giao diện Gradio so sánh trực quan ba mô hình.

Tài liệu tham khảo

1. Redmon, J., et al. “You Only Look Once: Unified, Real-Time Object Detection.” CVPR, 2016.
2. Kim, G., et al. “OCR-free Document Understanding Transformer (Donut).” ECCV, 2022.
3. Li, M., et al. “TrOCR: Transformer-based OCR with Pre-trained Models.” AAAI, 2023.
4. Ultralytics YOLO11 Documentation: <https://docs.ultralytics.com>
5. Hugging Face Transformers: <https://huggingface.co/docs/transformers>
6. Repository dự án: https://github.com/zangdo/Captcha_Reader

Phụ lục A

Phụ lục: Hướng dẫn cài đặt và chạy

A.1 Cài đặt môi trường

```
1 pip install uv
2 uv sync
3 # Copy modeling_donut_swin.py vào transformers (xem README)
4 # Tải dataset + weights từ Google Drive
```

A.2 Lệnh chạy chính

Lệnh	Mô tả
uv run sample.py	Xem pipeline sinh + augment
uv run generate_dataset.py	Sinh/convert dataset
uv run YOLO.py	Train YOLO
uv run Donut_train.py	Train Donut
uv run TrOCR_train.py	Train TrOCR
uv run app.py	Demo Gradio